

Chapter 28 : Matrix Operations

1) **Numerical stability**: Due to the limited precision of floating-point representations in actual computers, round-off errors in numerical computations may become amplified over the course of a computation, leading to incorrect results; such computations are numerically unstable.

2) We use uppercase letters to denote matrices and corresponding subscripted lowercase letters to denote their elements eg $A = (a_{ij})$.

The set of all $m \times n$ matrices with entries drawn from a set S is denoted by $S^{m \times n}$. The set of all $m \times n$ matrices with real valued entries is denoted by $\mathbb{R}^{m \times n}$.

The transpose of a matrix A is denoted A^T and is obtained by exchanging the rows and columns of A .

A vector is a one dimensional array of numbers. We use lowercase letters to denote vectors and we denote the i th element of a size- n vector by x_i , for $i = 1, 2, \dots, n$. The standard form of a vector is a column vector equivalent to an $n \times 1$ matrix. The corresponding row vector is obtained by taking the transpose x^T .

The unit vector e_i is the vector whose i th element is 1 and all of whose other elements are 0.

A zero matrix is a matrix whose every entry is 0. Such a matrix is often denoted O . Usually the size of a

unit vector or zero matrix is clear from the context.

An $n \times n$ matrix is called a square matrix. Several special cases of square matrix are of particular interest:-

① **Diagonal matrix**: All of the off-diagonal elements are zero i.e. $a_{ij} = 0$ whenever $i \neq j$. The matrix can be specified by ^{listing} the elements along the diagonal:

$$\text{diag}(a_{11}, a_{22}, \dots, a_{nn}) = \begin{bmatrix} a_{11} & & & 0 \\ 0 & a_{22} & & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{bmatrix}$$

② **Identity matrix**: The $n \times n$ identity matrix I_n is a diagonal matrix with 1's along the diagonal.

$$I_n = \text{diag}(1, 1, \dots, 1) = \begin{bmatrix} 1 & & & 0 \\ 0 & 1 & & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

When I appears without a subscript, its size can be derived from the context. Every column of an identity matrix is a unit vector. The i th column of an identity matrix is the unit vector e_i .

③ **Tridiagonal matrix**: A tridiagonal matrix T is one for which $t_{ij} = 0$ if $|i-j| > 1$. Nonzero entries appear only on the main diagonal, immediately above the main diagonal ($t_{i, i+1}$ for $i = 1, 2, \dots, n-1$) or immediately below the main diagonal ($t_{i+1, i}$ for $i = 1, 2, \dots, n-1$).

④ **Upper-triangular matrix:** An upper-triangular matrix U is one for which $u_{ij} = 0$ if $i > j$. All entries below the diagonal are zero:

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & \dots & u_{1n} \\ 0 & u_{22} & & & \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & & & u_{nn} \end{bmatrix}$$

An upper-triangular matrix is said to be **unit upper-triangular** if it has all 1's along the diagonal.

⑤ **Lower-triangular matrix:** A lower-triangular matrix L is one for which $l_{ij} = 0$ if $i < j$. All entries above the diagonal are zero:

$$L = \begin{bmatrix} l_{11} & & & & 0 \\ & l_{22} & & & 0 \\ & & \ddots & & \vdots \\ & & & l_{nn} & \\ l_{1n} & l_{2n} & \dots & l_{nn} \end{bmatrix}$$

A lower-triangular matrix is said to be **unit lower-triangular** if it has all 1's along the diagonal.

⑥ **Permutation matrix:** A permutation matrix P has exactly one 1 in every row or column, and 0's elsewhere, eg:-

$$P = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

A permutation matrix is called so because multiplying a vector x by a permutation matrix has the effect of permuting (rearranging) the elements of x .

⑦ **Symmetric matrix:** A matrix is said to be symmetric if it satisfies the condition $A = A^T$.

eg:

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 6 & 4 \\ 3 & 4 & 5 \end{pmatrix}$$

3) Operations on matrices

→ The elements of a matrix or vector are numbers from a number system, such as real numbers, complex numbers, or integers modulo a prime.

→ If λ is a number and $A = (a_{ij})$ is a matrix, then $\lambda A = (\lambda a_{ij})$ is the scalar multiple of A .

Two matrices A and B are said to be compatible for multiplication if the number of columns in A equals the number of rows in B . If $A = (a_{ij})$ is an $m \times n$ matrix and $B = (b_{jk})$ is an $n \times p$ matrix, then their matrix product $C = AB$ is an $m \times p$ matrix $C = (c_{ik})$, where

$$c_{ik} = \sum_{j=1}^n a_{ij} b_{jk} \quad \text{for } i = 1, 2, \dots, m \text{ and } k = 1, 2, \dots, p$$

The simple procedure for matrix multiplication runs in time $O(n^3)$, performing n^3 multiplications and $n^2(n-1)$ additions.

→ Matrices have many properties which are typical of numbers:-

① Identity matrices are identities for matrix multiplication:-

$$I_n A = A I_n = A \text{ for } n \times n \text{ matrix } A.$$

② $A 0 = 0$. Multiplication with zero-matrix gives zero-matrix.

③ Matrix multiplication is associative:-

$$A(BC) = (AB)C$$

④ Matrix multiplication is distributive w.r.t. addition:-

$$A(B+C) = AB + AC$$

$$(B+C)D = BD + CD$$

⑤ For $n > 1$, multiplication of $n \times n$ matrices is not commutative:-

$$AB \neq BA \text{ for } n \times n \text{ matrices } A \text{ and } B.$$

⑥ If A is an $n \times n$ matrix and x is a vector of size n , then Ax is a vector of size n .

⑦ Inner product of vectors: If x and y are vectors of size n , then

$$x^T y = \sum_{i=1}^n x_i y_i$$

is a number called the inner product of x and y (actually a 1×1 matrix).

⑧ Outer product of vectors: If x and y are vectors of

size- n , then the matrix xy^T is an $n \times n$ matrix Z called the outer product of x and y .

$$Z_{ij} = x_i y_j$$

Now we

⑧ **Norm of a vector:** The (euclidean) norm $\|x\|$ of a vector x of size n is defined by

$$\begin{aligned}\|x\| &= \sqrt{x_1^2 + x_2^2 + \dots + x_n^2} \\ &= \sqrt{x^T x}\end{aligned}$$

Thus, the norm of a vector x is its length in n -dimensional euclidean space.

→ The inverse of an $n \times n$ matrix A is the $n \times n$ matrix A^{-1} such that $AA^{-1} = I_n = A^{-1}A$.

Many nonzero $n \times n$ matrices do not have inverses. A matrix without an inverse is called noninvertible or singular. For example, $\begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}$.

A matrix that has an inverse is called invertible or nonsingular. If A and B are nonsingular $n \times n$ matrices, then

$$(BA)^{-1} = A^{-1}B^{-1}.$$

Inverse operation commutes with the transpose operation:

$$(A^{-1})^T = (A^T)^{-1}$$

show this

→ The vectors $x_1, x_2, \dots, x_n$ are linearly dependent if there exist coefficients $c_1, c_2, \dots, c_n$ not all zero, such that $c_1 x_1 + c_2 x_2 + \dots + c_n x_n = 0$. Otherwise the vectors are said to be linearly independent.

→ Some subset of the set of columns of any matrix can be linearly independent. The **column rank** of a nonzero $m \times n$ matrix A is the size of the largest set of linearly independent columns of A . The **row rank** of A is the size of the largest set of linearly independent rows of A .

The row rank of any matrix always equals its column rank, so that we can simply refer to the "rank" of a matrix. The rank of an $m \times n$ matrix is an integer between 0 and $\min(m, n)$, inclusive. The rank of a zero matrix is 0, and the rank of an $n \times n$ identity matrix is n .

An equivalent and often more useful definition is that the rank of a nonzero $m \times n$ matrix A is the smallest number r such that there exist matrices B and C of ~~positive~~ sizes $m \times r$ and $r \times n$ such that:—

$$A = BC$$

→ A square $n \times n$ matrix is said to have "full rank" if its rank is n . An $m \times n$ matrix has "full rank" if its rank is $\min(m, n)$.

Theorem:

A square matrix has full rank if and only if it is non-singular.

→ A **null vector** for a matrix A is a nonzero vector x such that $Ax = 0$.

~~This means:~~

Theorem:

A matrix A has full column rank if and only if it does not have ~~any~~ null vectors.

Corollary:

A square matrix A is singular if and only if it has a null vector.

→ The i th "minor" of an $n \times n$ matrix A , is the $(n-1) \times (n-1)$ matrix $A_{[i]}$ obtained by deleting the i th row and j th column of A . The "determinant" of an $n \times n$ matrix A can be defined recursively in terms of its minors by:

$$\det(A) = \begin{cases} a_{11} & , \text{if } n=1 \\ \sum_{j=1}^n (-1)^{i+j} a_{ij} \det(A_{[i]}) & , \text{if } n \geq 2, \text{ for any fixed } i=1, \dots, n \end{cases}$$

The term $(-1)^{i+j} \det(A_{[i]})$ is known as the "cofactor" of the element a_{ij} .

→ Theorem (Determinant properties)

The determinant of any square matrix A has the following properties:-

- 1) If any row or any column of A is zero, then $\det(A) = 0$.
- 2) The determinant of A is multiplied by λ if the entries of any one row or any one column of A are all multiplied by λ .
- 3) The determinant of A remains unchanged if the entries in one row are added to respective entries of another row.
- 4) The determinant of A equals the determinant of A^T .

5) The determinant of A gets multiplied by -1 , if any two rows or any two columns are interchanged.

For any 2 square matrices A and B we have:

$$\det(AB) = \det(A) \det(B)$$

Theorem:

An $n \times n$ matrix A is singular if and only if $\det(A) = 0$.

→ $(AB)^T = B^T A^T$ and $A^T A$ is always symmetric.

Positive-definite matrices

Any $n \times n$ matrix A is positive definite if $x^T A x > 0$ for all size- n vectors $x \neq 0$.

For example, the identity matrix is positive definite for any non-zero vector x :-

$$\begin{aligned} x^T I_n x &= x^T x \\ &= \sum_{i=1}^n x_i^2 > 0. \end{aligned}$$

Theorem: For any matrix A with full column rank, the matrix $A^T A$ is positive-definite.

Proof: For any vector x (non-zero) :-

$$x^T (A^T A) x = (Ax)^T (Ax) = \|Ax\|^2 \geq 0$$

$\|Ax\| = 0$ implies $Ax = 0$. Since A has full column rank, $Ax = 0$ implies $x = 0$. Hence $A^T A$ is positive definite.

4) Solving systems of linear equations by LUP decomposition:

You have n linear equations in n unknowns:-

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$\vdots$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

$$\Rightarrow \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

$$\Rightarrow A x = b$$

If A is nonsingular :-

$$x = A^{-1}b$$

- The method of multiplying A^{-1} with b suffers from numerical instability. LUP decomposition on the other hand is numerically stable and is faster in practice.
- If A has full rank i.e. rank equals the number of unknowns n , A is nonsingular and unique solution exists. If rank of A is less than n the system is underdetermined. Such systems have infinitely many solutions or no solution at all if the equations are inconsistent. If the no. of equations exceeds the no. of unknowns the system is overdetermined and there may not be any solution.

→ In LUP decomposition you find three $n \times n$ matrices L, U and P such that:-

$$PA = LU$$

where,

P is a permutation matrix

L is a unit lower-triangular matrix

U is an upper-triangular matrix

The matrices L, U and P are then said to form an LUP decomposition of the "nonsingular" matrix A . Every nonsingular matrix possesses such a decomposition.

Having found an LUP decomposition of A , we use "forward substitution" and then "back substitution" to find the solution vector x .

$$Ax = b$$

$$\Rightarrow PAx = Pb$$

$$\Rightarrow LUx = Pb$$

$$\text{let } Ux = y$$

$$\text{then, } Ly = Pb$$

This is a triangular linear system which is solved for using forward substitution:-

Forward substitution solves the lower triangular system in $\mathcal{O}(n^2)$ time. The permutation matrix P is represented compactly by the array $\pi[1..n]$. If $\pi[i] = k$, it implies that the k th column of i th row is 1. i.e. $P_{i,\pi[i]} = 1$.

$Lx = Pb$ can be written as:-

$$\begin{aligned} y_1 &= b_n[1] \\ l_{21}x_1 + x_2 &= b_n[2] \\ l_{31}x_1 + l_{32}x_2 + x_3 &= b_n[3] \\ &\vdots \\ l_{n1}x_1 + l_{n2}x_2 + \dots + x_n &= b_n[n] \end{aligned}$$

yielding:- $x_1 = b_n[1]$

$$x_2 = b_n[2] - l_{21}x_1$$

$$x_i = b_n[i] - \sum_{j=1}^{i-1} l_{ij}x_j \quad \left[\begin{array}{l} \text{To solve for } x_i \text{ substitute} \\ x_1, x_2, \dots, x_{i-1} \text{ forward} \\ \text{into the } i\text{th equation.} \end{array} \right]$$

Having solved for vector x we solve for vector u by back substitution.

Back substitution solves the upper triangular system in $\Theta(n^2)$ time.

$Ux = y$ can be written as:-

$$u_{11}x_1 + u_{12}x_2 + \dots + u_{1n}x_n = y_1$$

$$u_{22}x_2 + \dots + u_{2n}x_n = y_2$$

... ..

$$u_{n,n-1}x_{n-1} + u_{nn}x_n = y_{n-1}$$

$$u_{nn}x_n = y_n$$

yielding:-

$$x_n = y_n / u_{nn}$$

$$x_{n-1} = (y_{n-1} - u_{n,n-1}x_n) / u_{n,n-1}$$

$$x_i = \left(\gamma_i - \sum_{j=i+1}^n u_{ij} x_j \right) / u_{ii}$$

→ given P, L, U and b the procedure LUP-SOLVE values for x by forward and then back substitution

LUP-SOLVE (L, U, P, b)

```

1  n ← rows[L]
2  for i ← 1 to n
3      do  $\gamma_i \leftarrow b_{n[i]} - \sum_{j=1}^{i-1} l_{ij} \gamma_j$ 
4  for i ← n down to 1
5      do  $x_i \leftarrow (\gamma_i - \sum_{j=i+1}^n u_{ij} x_j) / u_{ii}$ 
6  return x

```

Computing an LU decomposition:

If A is an $n \times n$ nonsingular matrix and $P = I_n$ we can find a factorization $A = LU$. The matrices L and U then form an "LU decomposition" of A .

We perform LU decomposition by Gaussian elimination. We start by subtracting multiples of the first eqⁿ. from the other eqⁿs so that the 1st variable is removed from those eqⁿs. Then we subtract multiples of the 2nd eqⁿ. from the 3rd and subsequent eqⁿs, so that now the 1st 2 variables are removed from them. We continue the procedure until the system that is left has an upper-

triangular form and in fact is the matrix U . The matrix made up of the row multipliers that cause variables to be eliminated is the matrix L .

We devise a recursive algorithm to implement this strategy. We wish to construct an LU decomposition for an $n \times n$ nonsingular matrix A . If $n=1$ we are done since $L=I_1$ and $U=A$. If $n>1$, we break A into 4 parts:-

$$A = \left(\begin{array}{c|cccc} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & & & \\ \vdots & & & \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{array} \right) = \begin{pmatrix} a_{11} & w^T \\ v & A' \end{pmatrix}$$

v is a size- $(n-1)$ column vector

w^T is a size- $(n-1)$ row vector

A' is an $(n-1) \times (n-1)$ matrix

$$= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix}$$

$\hookrightarrow 0$ vector

$A' - vw^T/a_{11}$ is called the Schur complement of A with respect to a_{11} .

$\rightarrow$ If A is nonsingular then the Schur complement is also nonsingular. Suppose the Schur complement which is an $(n-1) \times (n-1)$ matrix is singular. Then by some theorem its row rank is strictly less than $(n-1)$.

The matrix $\begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix}$ must have a row

rank strictly less than $n-1$. [Since the bottom $(n-1)$ entries of 1st column are 0]. The row rank of this is

entire matrix, therefore, is strictly less than n and we derive a contradiction that A is singular.

Since the Schur complement is nonsingular it must also have an LU decomposition, say

$$A' - v w^T / a_{11} = L' U'$$

then,

$$A = \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - v w^T / a_{11} \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & L' U' \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ v/a_{11} & L' \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & U' \end{pmatrix}$$

$$= L U$$

→ This method does not work if $a_{11} = 0$ because it then divides by 0. In general, it does not work if any of the diagonal elements is 0. The elements by which we divide during LU decomposition are called ~~transformed~~ pivots. The reason we include a permutation matrix P is that it allows us to avoid division by zero elements. Using permutations to do so is called "pivoting".

The symmetric positive definite matrices require no pivoting since for them $P = I_n$.

→ The procedure LU-DECOMPOSITION converts a tail recursive procedure into an iterative one. It does not fill the entries of the matrix U below the diagonal or the entries of the matrix L

on and above the diagonal.

LU-DECOMPOSITION(A)

```
1  n ← rows[A]
2  for k ← 1 to n
3      do ukk ← akk
4          for i ← k+1 to n
5              do lik ← aik/ukk
6                  uki ← aki
7          for i ← k+1 to n
8              do for j ← k+1 to n
9                  do aij ← aij - likukj
10 return L and U
```

$A = \begin{pmatrix} 1 & 0 \\ \sqrt{a_{21}} & L' \end{pmatrix} \begin{pmatrix} a_{21} & W^T \\ 0 & U' \end{pmatrix}$

> Schur's complement for numerical processing

LU-DECOMPOSITION runs in time $\Theta(n^3)$.

→ A standard optimization to the above procedure is to hold the ^{significant} values of L and U "in place" in the matrix A.

Thus $a_{ij} = l_{ij}$ if $i > j$ and $a_{ij} = u_{ij}$ if $i \leq j$.

Computing an LUP decomposition:

→ In solving a system of linear equations $Ax=b$, we must pivot on off-diagonal elements of A to avoid division by 0. Division by ^{any small value} ~~zero~~ is also undesirable because numerical instabilities may result. We therefore pivot on the largest element of a column.

→ If a_{k1} be the largest element of the 1st column we exchange rows 1 and k to let a_{k1} be the pivot. This is equivalent to multiplying the matrix A with a permutation matrix Q such that

$$QA = \begin{pmatrix} a_{k1} & w^T \\ v & A' \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix}$$

If A is nonsingular, then the schur complement $A' - vw^T/a_{k1}$ is also nonsingular and can be written as:-

$$P'(A' - vw^T/a_{k1}) = L'U'$$

Let us define $P = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} Q$. This is a permutation matrix since product of 2 permutation matrices is always a permutation matrix.

$$\begin{aligned} PA &= \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} Q A \\ &= \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & P' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & P(A' - vw^T/a_{k1}) \end{pmatrix} \end{aligned}$$

$$= \begin{pmatrix} 1 & 0 \\ p'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & L'U' \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ p'v/a_{k1} & L' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & U' \end{pmatrix}$$

→ The procedure LUP-DECOMPOSITION exploits tail-recursion with iteration. The permutation matrix is maintained as the array $\pi[1..n]$. L and U are stored in place in matrix A . Thus, at procedure completion:-

$$a_{ij} = \begin{cases} l_{ij} & \text{if } i > j \\ u_{ij} & \text{if } i \leq j \end{cases}$$

LUP-DECOMPOSITION(A)

```

1  n ← rows[A]
2  for i ← 1 to n
3      do  $\pi[i] \leftarrow i$ 
4  for k ← 1 to n
5      do  $p \leftarrow 0$       ▷  $p$  holds the magnitudelargest element of a col.
6      for i ← k to n
7          do if  $|a_{ik}| > p$ 
8              then  $p \leftarrow |a_{ik}|$ 
9                   $k' \leftarrow i$ 
10     if  $p = 0$ 
11         then error "singular matrix"
12     exchange  $\pi[k] \leftrightarrow \pi[k']$ 
13     for i ← 1 to n
        do exchange  $a_{ki} \leftrightarrow a_{k'i}$ 

```

```

15 for  $i \leftarrow k+1$  to  $n$ 
16     do  $a_{ik} \leftarrow a_{ik}/a_{kk}$ 
17     for  $j \leftarrow k+1$  to  $n$ 
18         do  $a_{ij} \leftarrow a_{ij} - a_{ik}a_{kj}$ 

```

LUP-DECOMPOSITION runs in time $\Theta(n^3)$ which is same as that of LU decomposition. Thus, pivoting costs us only a constant factor in time.

5) Inverting matrices

Computing a matrix inverse from LUP decomposition:

The equation $AX = I_n$ defines the matrix X as the inverse of matrix A . It can be viewed as a set of n distinct equations of the form $Ax = b$, which can be solved by LUP decomposition for A . Let x_i denote the i th column of X . The i th column of I_n is the unit vector e_i .

Thus, $AX_i = e_i$

which can be solved for x_i using LUP decomposition of A .

The computation of the inverse X takes $\Theta(n^3)$ time since each of the n columns x_i can be found in time $\Theta(n^2)$.

Matrix multiplication and matrix inversion:

Matrix multiplication and matrix inversion are equivalently hard problems in that we can use an algorithm for one to solve the other in asymptotic running time.

If $M(n)$ denotes the time to multiply two $n \times n$ matrices, then there is a way to invert an $n \times n$ matrix in time $O(M(n))$. If $I(n)$ denotes the time to invert an $n \times n$ matrix, then there is a way to multiply two $n \times n$ matrices in time $O(I(n))$.

Theorem (Multiplication is no harder than inversion):

If we can invert an $n \times n$ matrix in time $I(n)$, where $I(n) = \Omega(n^2)$, and $I(n)$ satisfies the regularity condition $I(3n) = O(I(n))$, then we can multiply 2 $n \times n$ matrices in time $O(I(n))$.

Proof: Let A and B be two $n \times n$ matrices whose matrix product we wish to compute. Define a $3n \times 3n$ matrix D by:-

$$D = \begin{pmatrix} I_n & A & 0 \\ 0 & I_n & B \\ 0 & 0 & I_n \end{pmatrix}$$

The inverse of D is:-

$$D^{-1} = \begin{pmatrix} I_n & -A & AB \\ 0 & I_n & -B \\ 0 & 0 & I_n \end{pmatrix}$$

By taking the upper right $n \times n$ submatrix of D^{-1} we can compute AB .

We can construct D in $\Theta(n^2) = O(I(n))$ time.
We can invert D in $O(I(n)) = O(I(n))$ time [by the regularity condition on $I(n)$]. We thus have $M(n) = O(I(n))$.

$I(n)$ would satisfy the regularity condition whenever $I(n) = \Theta(n^c \lg^d n)$ where $c > 0$ and $d \geq 0$.

Theorem: Inversion is no harder than multiplication.

→ We can thus use Strassen's algorithm for matrix multiplication to invert a matrix.

6) Symmetric positive definite matrices:

Lemma: Any positive definite matrix is nonsingular.

Proof: Suppose that a matrix A is singular. Then by some earlier proved corollary, there exists a nonzero vector x such that $Ax = 0$. Hence, $x^T Ax = 0$, and A cannot be positive definite.

→ The k th leading submatrix of A is the matrix A_k consisting of the intersection of the 1st k rows and 1st k columns of A .

Lemma: If A is a symmetric positive-definite matrix, then every leading submatrix of A is symmetric and positive definite.

Proof: That each leading submatrix A_k is symmetric is obvious. Suppose that A_k is not positive definite. Then there exists a size- k vector $x_k \neq 0$ such that

$x_k^T A_k x_k \leq 0$. If the size of A is $n \times n$, we define the size- n vector x as $x = (x_k^T \ 0)^T$, where there are $n-k$ 0's following x_k . Then we have:-

$$\begin{aligned} x^T A x &= (x_k^T \ 0) \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \begin{pmatrix} x_k \\ 0 \end{pmatrix} \\ &= (x_k^T \ 0) \begin{pmatrix} A_k x_k \\ B x_k \end{pmatrix} \\ &= x_k^T A_k x_k \\ &\leq 0 \end{aligned}$$

which contradicts A being positive definite.

→ $A = \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix}$. The Schur complement of A with respect to A_k is defined as

$$S = C - B A_k^{-1} B^T.$$

Since A_k is positive definite (by previous lemma), A_k^{-1} exists and S is well defined.

lemma (Schur complement lemma): If A is a symmetric positive-definite matrix and A_k is a leading $k \times k$ submatrix of A , then the Schur complement of A with respect to A_k is symmetric and positive definite.

Corollary: LU decomposition of a symmetric positive-definite matrix never causes a division by zero.

Proof: Let A be a symmetric positive-definite matrix. We shall prove something stronger than the statement of the corollary: every pivot is strictly

positive. The first pivot is a_{11} . $a_{11} = e_1^T A e_1$ where e_1 is the first unit vector. But $e_1^T A e_1 \geq 0$, since A is positive-definite. Hence $a_{11} > 0$.

Since the Schur complement of A with respect to a_{11} is also positive definite, the Schur complement lemma implies that every pivot is positive by induction.

7) Least square approximation: An application of symmetric positive-definite matrices

→ Fitting curves to given sets of data points is an important application of symmetric positive-definite matrices. Suppose that we are given a set of m data points:-

$(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$ where the y_i are known to be subject to measurement errors.

→ We want to determine a function $F(x)$ such that

$$y_i = F(x_i) + \eta_i \quad i = 1, 2, \dots, m$$

where, the approximation errors η_i are small.

→ The form of the function F depends upon the problem at hand. Here, we assume that it has the form of a linearly weighted sum

$$F(x) = \sum_{j=1}^n c_j f_j(x) \quad : \text{ (small errors) }$$

The f_j 's are called "basis functions". The c_j 's of summands n and the basis functions f_j depend upon the problem at hand.

A common choice is $f_j(x) = x^{j-1}$, which means that

$$F(x) = C_1 + C_2x + C_3x^2 + \dots + C_nx^{n-1}.$$

which is a polynomial of degree $n-1$ in x .

If we choose $n = m$, we can calculate each

y_i "exactly" in the equation $y_i = F(x_i) + \epsilon_i$. Such a high-degree F "fits the noise" as well as the data, however, and generally gives poor results when used to predict y for previously unseen values of x .

It is generally